Module 1: Introduction to GenAI & LLMs

- What is GenAI?
- LLM Fundamentals
- Labs: Hands-on LLM Fundamentals

What is Generative AI?

What is Generative AI?

Generative Artificial Intelligence (GenAI)

Generative + Artificial Intelligence

What is Generative AI?

Generative Artificial Intelligence (GenAI)

Generative

+

Artificial Intelligence

Computer programs capable of mimicking human intelligence

What is Generative AI?

Generative Artificial Intelligence (GenAI)

Generative

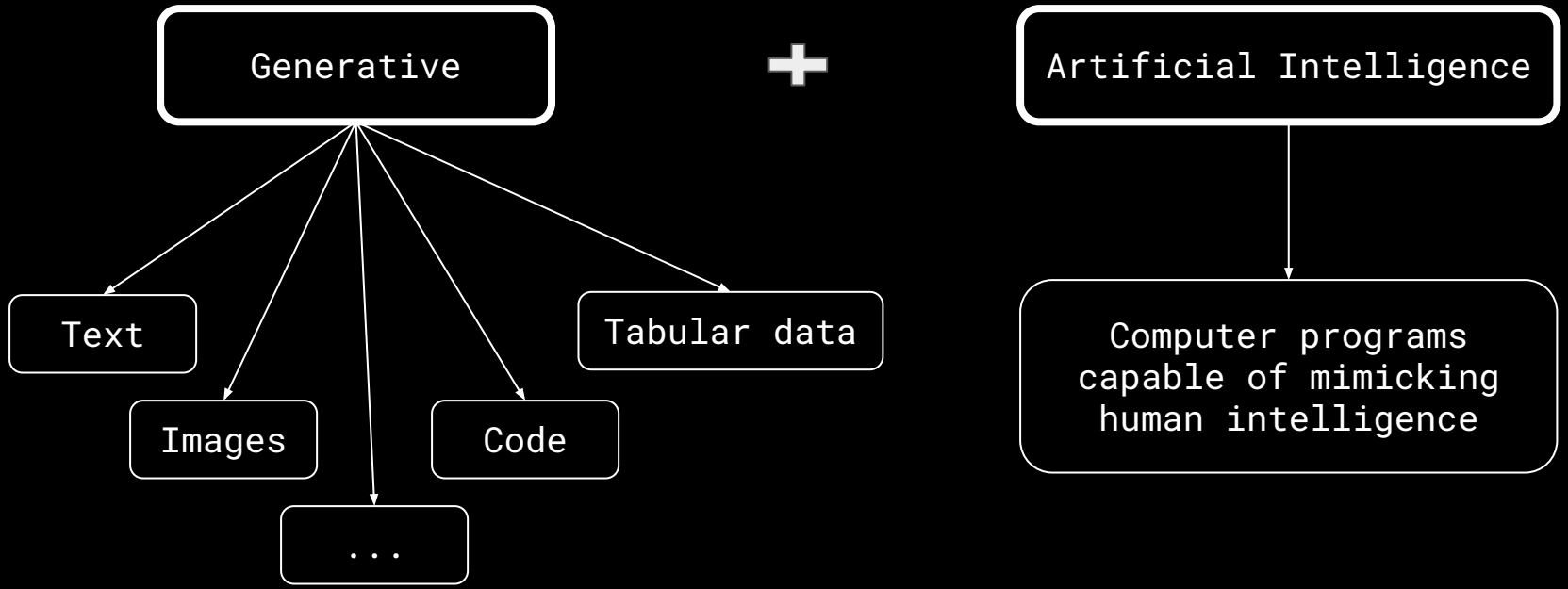
+

Artificial Intelligence

Ability to create original content, such as text, images, videos, code

What is Generative AI?

Generative Artificial Intelligence (GenAI)



Approaches of AI

Artificial Intelligence

Generative

Discriminative

Learn P(Y|X)

Understand the conditional distribution mapping features to output

Focuses on distinguishing between different classes.

Logistic regression, kNN, SVM

Approaches of AI

Artificial Intelligence

Generative

Learn P(X,Y)

Understand and capture the joint distribution of the data

Generate new data points similar to the original dataset

VAE, GANs, LLMs

Discriminative

Learn P(Y|X)

Understand the conditional distribution mapping features to output

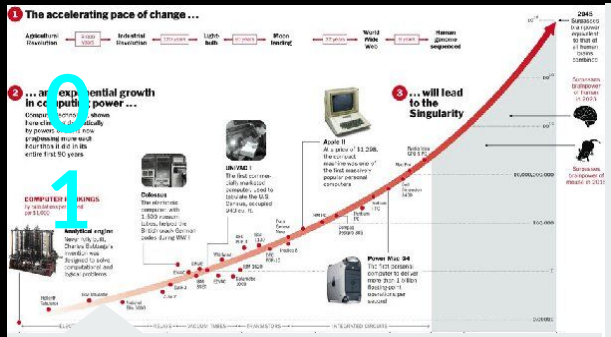
Focuses on distinguishing between different classes.

Logistic regression, kNN, SVM

Why Generative AI Now?

Why GenAI Now?

AI TRAINING



01

Growth in compute power

✅ Rise of specialized hardware (GPUs, TPUs)

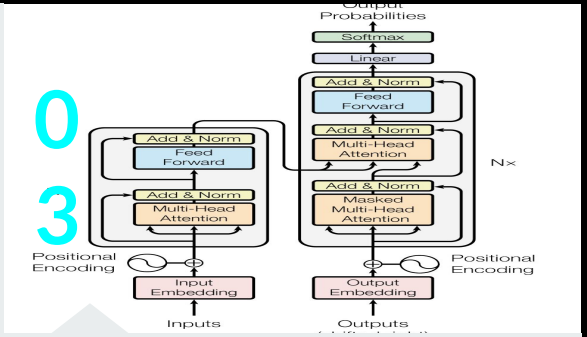
✅ Cloud computing

Large datasets

✅ Internet : images, videos, text

✅ Advances in data curation, high-quality sources.

02



03

Advanced Architectures: The Transformers

✅ Attention is All You Need

✅ Transformers power GPT, MISTRAL, LLAMA

Real-world Impacts

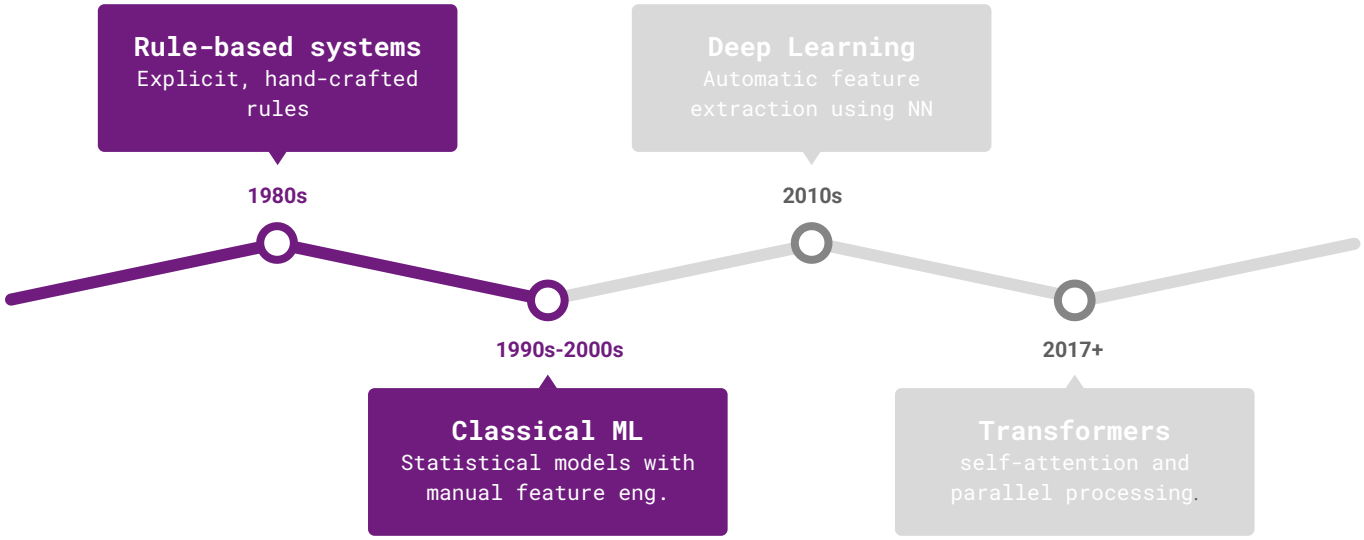
AI TRAINING

- ✅ Enhanced Productivity & Cost Reduction
- ✅ Automated Content Creation
- ✅ Personalized Marketing & Customer Engagement
- ✅ Product & Design Innovation

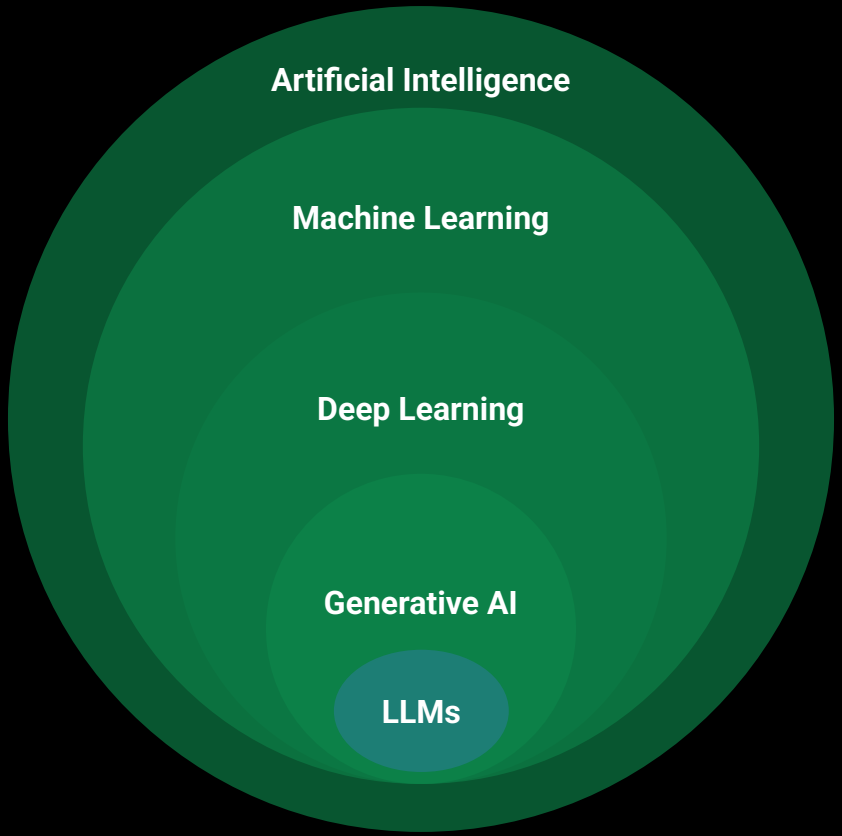
The Evolution of AI?

AI TRAINING

The Evolution of AI



The Evolution of AI



Classical ML vs. Deep Learning vs. LLMs

	Classical ML	Deep Learning	LLMs
Data requirements	Small	Large	Large
Feature Engineering	Manual	Automatic	Automatic
Interpretability	High	Lower	Lower
Typical Tasks	Fraud detection/customer segmentation	Image recognition/Machine Translation	chatbots/content generation

🎯 Conclusion – What is Generative AI?

🚀 Generative AI in a Nutshell:

- 🤖 AI that creates new content (text, images, music, code, etc.)
- 🧠 Powered by deep learning & probabilistic models

🔑 Key Takeaways:

- 🛠️ Learns from data to generate human-like outputs
- 📊 Probabilistic models (e.g., transformers) drive advancements
- ⚡ Applications span multiple domains (art, coding, medicine, etc.)

📌 Next Steps:

- 📖 Explore LLM Fundamentals
- 🛠️ Hands-on LLM Fundamentals

LLM Fundamentals

AI TRAINING

What is a Language Model?

What is a Language Model ?

- A language model is a probability distribution over sequences:

$$P_\theta\left(y\right) \tag{1}$$

$$y = \left(y_1, \ldots, y_T\right)$$

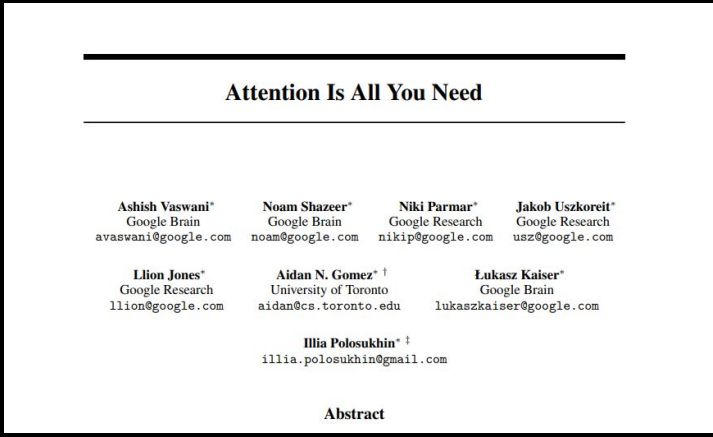
What is a Language Model ?

- A language model is a probability distribution over sequences:

$$P_\theta\left(y\right) \tag{1}$$

$$y = \left(y_1, \ldots, y_T\right)$$

$$\theta : transformer$$



Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com
Noam Shazeer* Google Brain noam@google.com
Niki Parmar* Google Research nikip@google.com
Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com
Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu
Łukasz Kaiser* Google Brain lukaszkaiser@google.com
Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

LM : Chain Rule

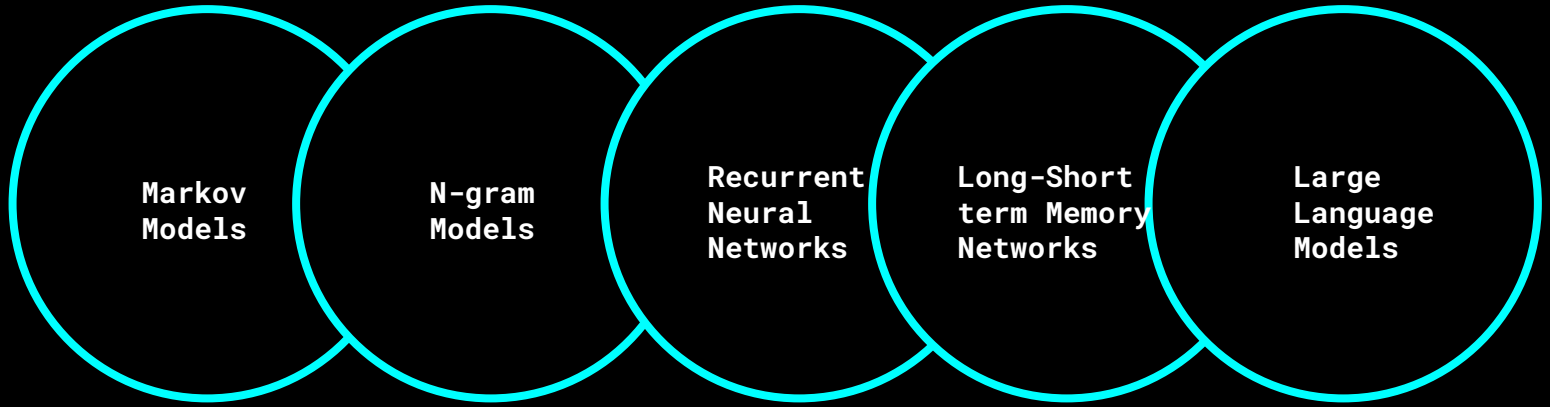
- Very often, LM factorizes (1) using chain rule

$$P_\theta\left(y\right) = \Pi_{t=1}^{T} P_\theta\left(y_t \mid y_{<t}\right) \tag{2}$$

- Typically, language models are autoregressive

Evolution of Language Models

Evolution of Language Models



Markov Models and N-Grams

Markov condition: The future is independent of the past, given the present.

$$P(y_n|y_1, y_2, \ldots, y_{n-1}) = P(y_n|y_{n-1})$$

N-Grams : Probability of a word depends on the previous N-1 words

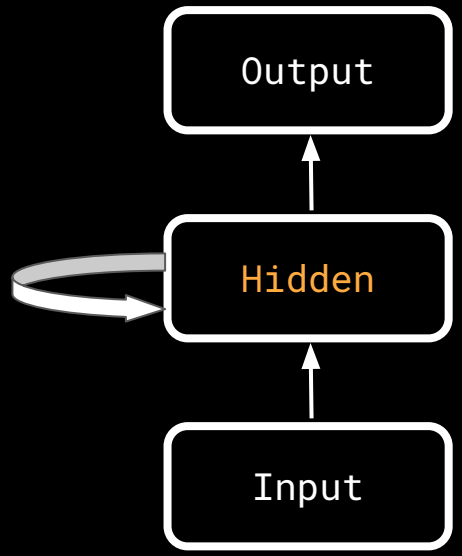
Examples: Unigram, Bigram, Trigram models

✅ Strengths: Easy to implement and interpret, smaller corpora and simpler tasks

❌ Limitations: Data sparsity, Limited context, OOV words

RNN

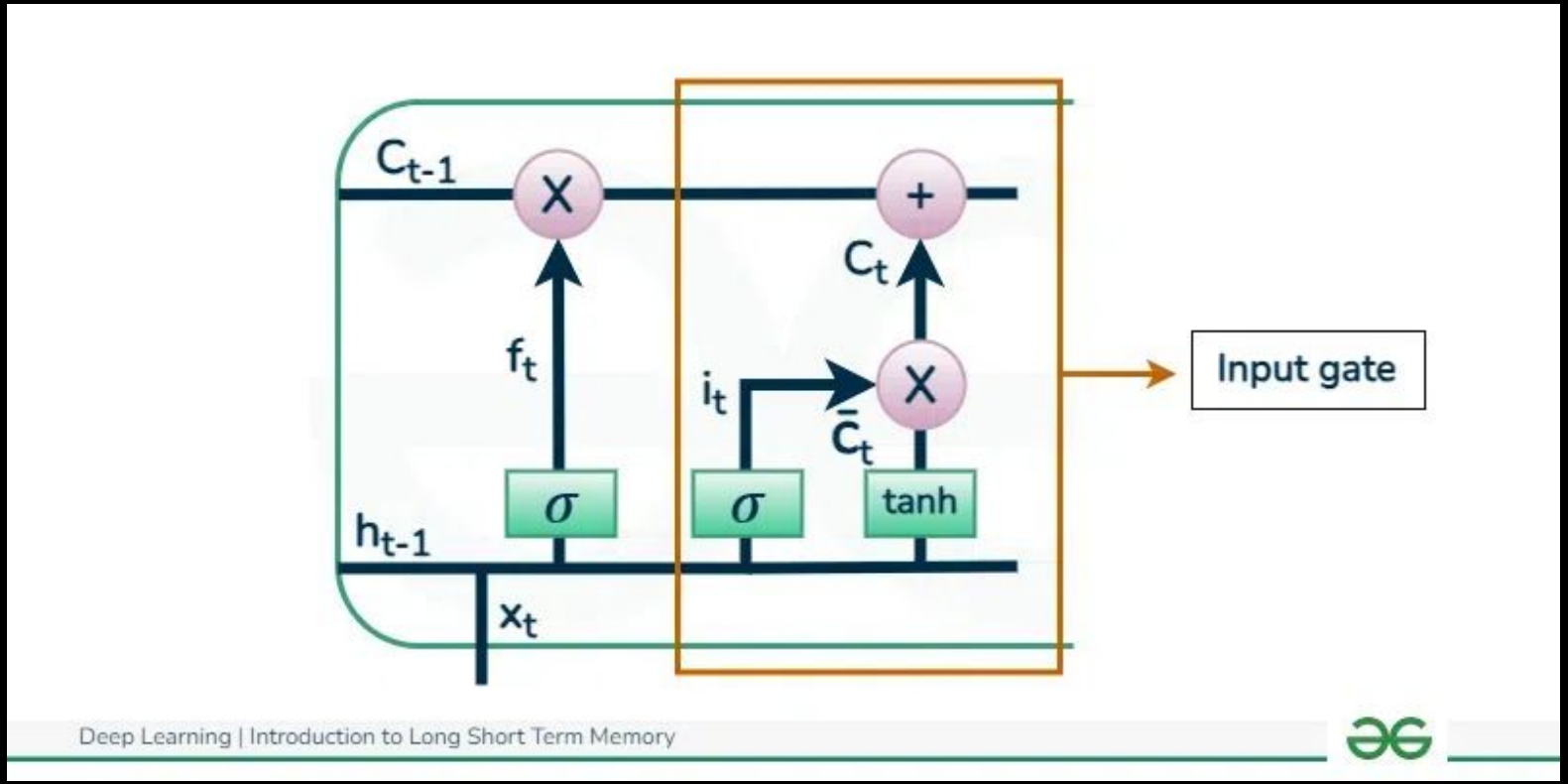
- To capture and retain things from the past, RNN introduces memory called hidden state



$$h_t = f(W_x x_t + W_h h_{t-1} + b)$$

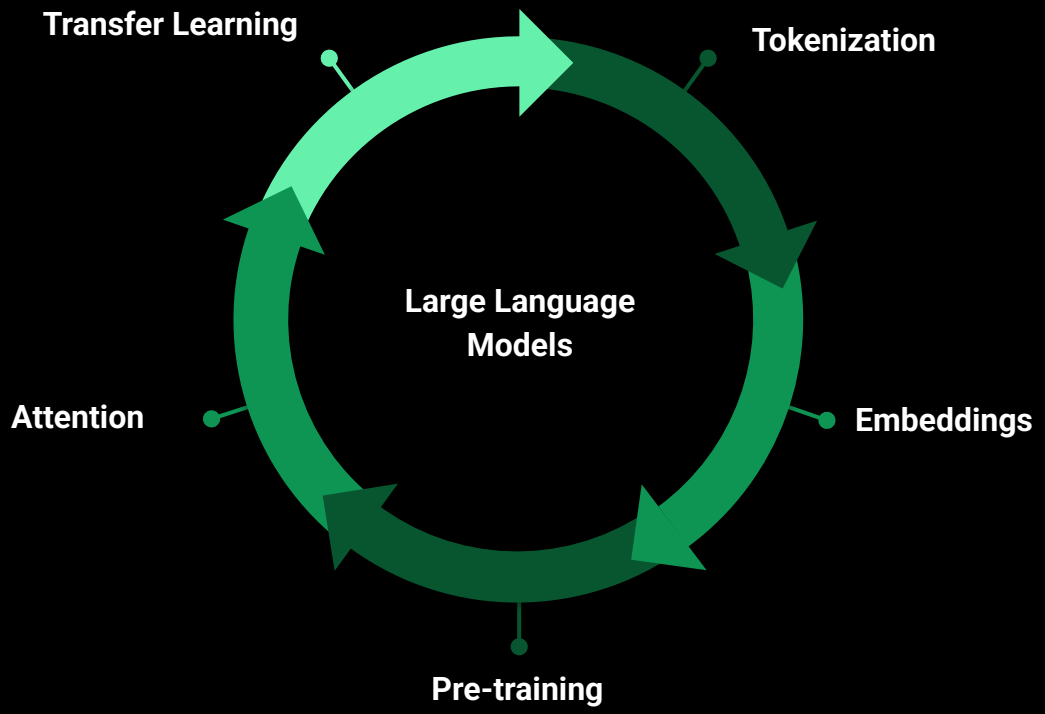
❌ Vanishing gradients, Exploding gradients, sequential processing

LSTM


C_{t-1}
X
+
C_t
f_t
i_t
X
$\bar{C}_t$
Input gate
σ
σ
tanh
h_{t-1}
x_t
Deep Learning | Introduction to Long Short Term Memory

Core Concepts

Core Concepts



Tokenization

Tokenization

📖 Definition:

- ✂️ Breaking text into smaller units (tokens)

🔍 Methods:

- 📌 Word-level (full words)
- 📌 Subword-level (BPE)
- 📌 Character-level (individual letters)

🎯 Benefits:

- ✅ Manages vocabulary size
- ✅ Handles out-of-vocabulary words
- ✅ Improves efficiency in text processing

Embeddings

AI TRAINING

🧐 What Are They?

Dense vector representations of tokens

🎯 Why Important?

✅ Capture semantic & syntactic relationships
✅ Reduce dimensionality vs. one-hot vectors

🚀 Popular Techniques:

📌 Word2Vec (CBOW, Skip-gram)
📌 GloVe
📌 BERT series, Gte, all-MiniLM-L6, text-embedding-ada, Bge

💪 Benefits:

✨ Boosts NLP performance
✨ Enables transfer learning

Attention Mechanism Overview

💡 Core Idea:

🧠 Models learn “where to look” in the input sequence

🔑 Key Steps:

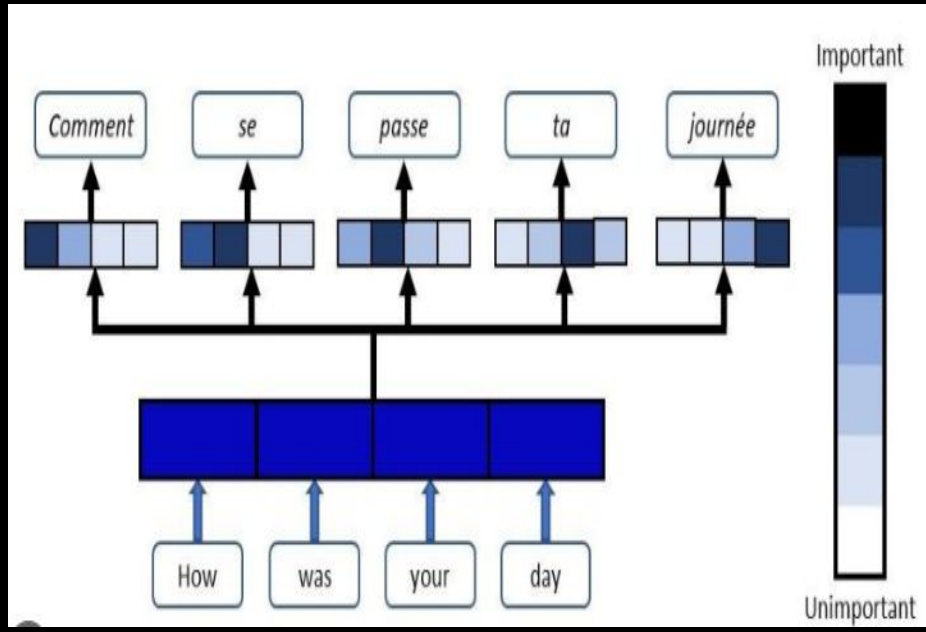
📊 Compute weights (relevance) for each token

➕ Create weighted sums of representations

🚀 Strengths

✅ Highly parallelizable

✅ Capture long-range dependencies more effectively



Why LLMs Matter

AI TRAINING

01	**Handling context at scale**	• Extended Context Windows • Reduced Fragmentation • Enhancing Understanding
02	**Transfer learning advantage**	• Pretraining on massive corpora • Fewer Task-Specific Examples • Wide applicability
03	**Real-Time Adaptation**	• Interactive Applications • Continual Learning • Immediate Feedback Loops

🎯 Conclusion

🚀 LLMs Transform NLP:

🔡 From tokenization to attention-based architectures

🔑 Key Takeaways:

🧩 Tokenization & embeddings form the foundation
✨ Attention mechanisms revolutionize context handling
🔄 Transfer learning & real-time adaptation drive modern applications

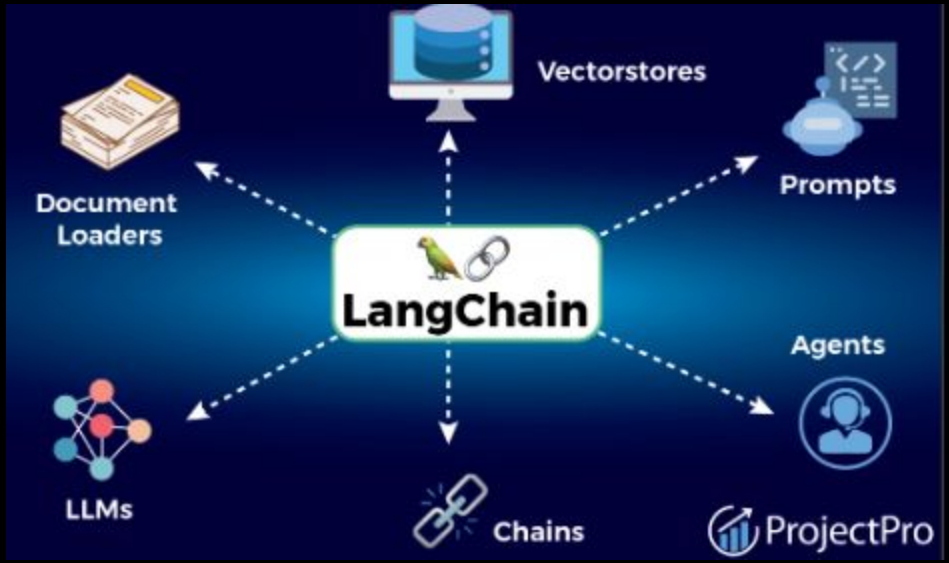
📌 Next Steps:
🛠️ Hands-on LLM Fundamentals

Mastering Langchain for AI Applications

AI TRAINING

What is LangChain?

LangChain is a powerful Python library and framework designed to empower developers in creating applications fueled by language models.



Core Components of LangChain

Core Components: Models

AI TRAINING

🧐 What Are They?

Abstract interface that wraps any generative engine, standardizing how you invoke it.

LLMs, Chat Models, and Text Embedding Models.

🎯 Why Important?

- ✅ Parameter Management (temperature, top_p, max_tokens etc)
- ✅ Invocation logic (retries, rate-limit handling, batching support)
- ✅ Output handling (text normalization, error detection)

Available Wrappers: OpenAI, AzureOpenAI, HuggingFaceHub, Anthropic, VertexAI etc

Core Components: Chains

AI TRAINING

🧐 What Are They?

Abstraction that encodes a sequence of calls—whether to a language model, a document retriever, another Chain, or a custom tool—exposing a simple run() interface

Reusability: chains can be composed, nested allowing you to build higher-level workflows from smaller building blocks.

🎯 Common Chain Patterns?

✅ LLMChain: wraps a single LLM call with a PromptTemplate for simple tasks
✅ SimpleSequentialChain: Executes a list of Chains in order
✅ RouterChain: classifies input intends and dynamically routes them to the appropriate sub-Chain

Core Components: Prompt Templates

AI TRAINING

🧐 What Are They?

help to translate user input and parameters into instructions for a language model.

Take as input a dictionary, where each key represents a variable in the prompt template to fill in.

✨ String PromptTemplates: Formats a single string prompt with named placeholders

```
from langchain_core.prompts import PromptTemplate

prompt_template = PromptTemplate.from_template("Tell me a joke about {topic}")

prompt_template.invoke({"topic": "cats"})
```

Core Components: Prompt Templates

AI TRAINING

✨ ChatPromptTemplates

Composes a sequence of messages (system/user/assistant) with placeholders, ideal for chat-based LLMs

```
from langchain_core.prompts import ChatPromptTemplate

prompt_template = ChatPromptTemplate([
    ("system", "You are a helpful assistant"),
    ("user", "Tell me a joke about {topic}")
])

prompt_template.invoke({"topic": "cats"})
```

This ChatPromptTemplate will construct two messages when called. The first is a system message, that has no variables to format. The second is a HumanMessage, and will be formatted by the topic variable the user passes in.

Core Components: Memory

Preserves context across calls, enabling coherent multi-turn interactions and stateful workflows

Buffer Memory: Stores the full conversation history in a rolling buffer

Windowed Memory: Keeps only the last n interactions to control token usage

```
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationChain
from langchain.llms import OpenAI

memory = ConversationBufferMemory()
chat = ConversationChain(llm=OpenAI(model_name="gpt-4"), memory=memory)
```